오즈 서버바인딩 가이드

2025-10-16

목차_Toc211523125

1. 개요 2

2. 환경 설정 4

3. 개발 소스 5

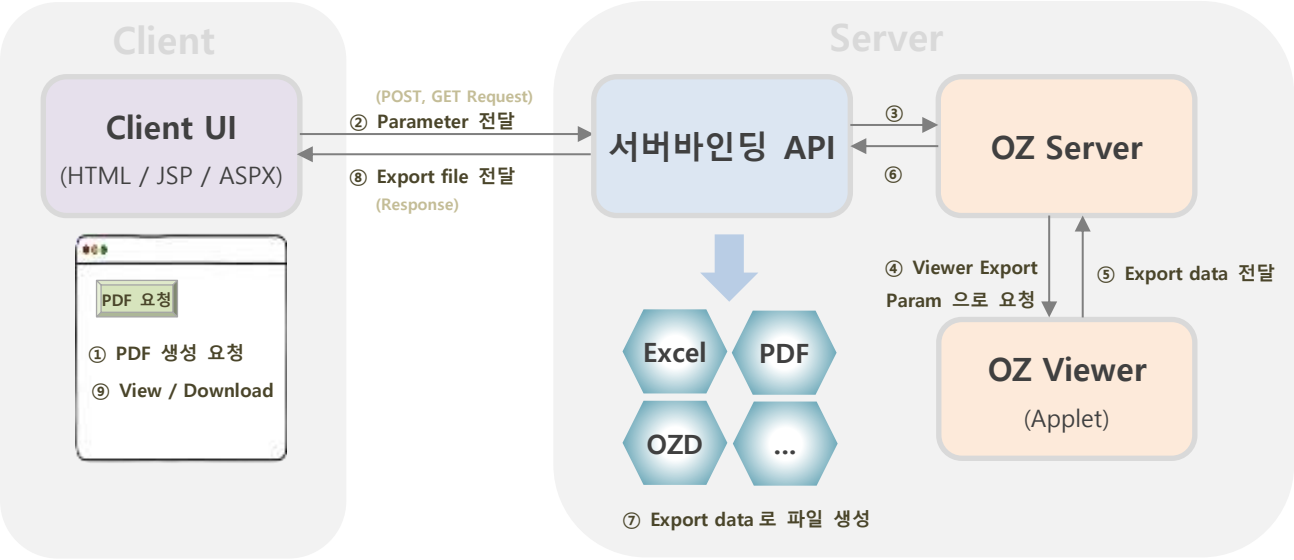
- 패러미터 설명 5
- 예제1 : PDF 생성 후 웹 브라우저에서 열기 6
- 예제2 : PDF 생성 후 서버에 저장 8
- 예제3 : 입력한 정보로 PDF 생성 11
- 예제4 : 멀티보고서 파일 생성 15
- 예제5 : 서버바인딩 서블릿 18

4. 디버깅 19

- 참고 사항 19
- 예제1 서버 로그 19
- OZ Service export serivce is not supported 에러 21
- export file key is not exist 에러 22

샘플소스 다운로드

1. 개요



* 서버바인딩 API 와 오즈 서버는 같은 웹 어플리케이션에 있어야 합니다.

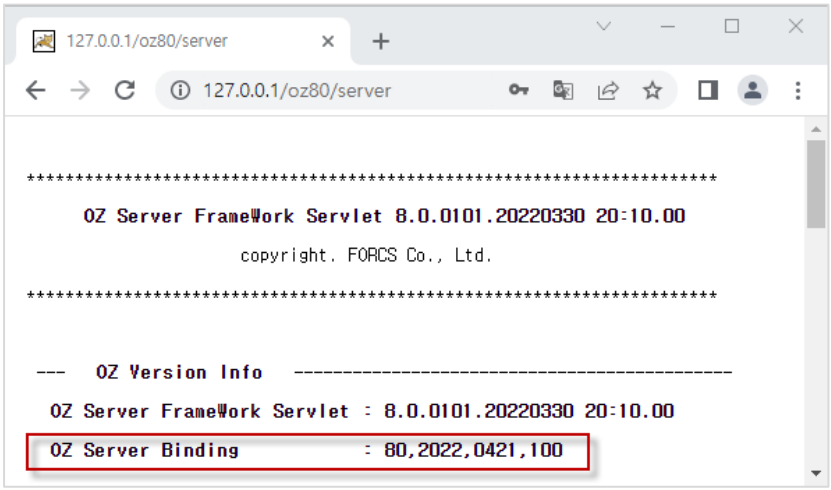
서버 바인딩이란 보고서를 서버에서 바인딩하여 다른 포맷 파일로 생성하는 기능입니다. 서버에서 바인딩되기 때문에 리포트 뷰어 설치나 미리보기가 따로 필요하지 않습니다.

지원하는 포맷은 PDF, XLS, XLSX, DOC, HWP, PPT, HTML, JPG, GIF, PNG, SVG, TIF, MHT, CSV, TXT 이 있습니다.

jsp나 aspx 페이지에서 get 또는 post 방식으로 보고서 패러미터를 전달하여 바로 PDF, XLS 등의 파일로 저장하거나 저장한 파일을 바로 웹 브라우저에 표시할 수 있습니다.

- 보고서를 오즈 서버에서 저장하기 때문에 <u>오즈 서버가 설치된 서버에 보고서에서 사용한 폰트가 설치되어 있어야 합니다.</u>
- 서블릿/닷넷 유형의 서버에서 제공합니다.
- 오즈 서버가 서블릿 타입인 경우 오즈 서버가 올라간 WAS의 자바 버전이 1.5 이상일 때 지원되며, 관련 라이브러리 파일(ozrresource.jar, ozviewer_lib.jar, poi-scratchpad-3.0.1.jar, uk.co.mmscomputing.imageio.tiff.jar)이 lib 폴더에 있어야 합니다. 기본 제공됩니다.
- 닷넷 타입 서버의 경우 Microsoft .NET Framework 3.5 이상이 설치된 환경에서만 가능하고, 서버 구동 시 관련 라이브러리 파일(OZWPFViewer51_Lib.dll)이 구동되어야 합니다.
- 웹 브라우저 주소 창에 서버 URL을 입력하면 오즈 서버 버전과 함께 서버 바인딩 뷰어의 버

전도 표시됩니다.


127.0.0.1/oz80/server
127.0.0.1/oz80/server
****
OZ Server FrameWork Servlet 8.0.0101.20220330 20:10.00
copyright. FORCS Co., Ltd.
****
--- OZ Version Info ----------
OZ Server FrameWork Servlet : 8.0.0101.20220330 20:10.00
OZ Server Binding : 80,2022,0421,100

2. 환경 설정

① 라이선스

서버바인딩 기능을 이용하려면 오즈 서버 라이선스에 USE-SERVERBIND="TRUE" 여야 합니다.

FALSE인 경우 영업대표를 통해 서버바인딩이 지원되는 라이선스로 발급받고, 적용합니다.

예) C:₩Tomcat₩webapps₩oz80₩WEB-INF₩license₩ozlicense.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<OZ-LICENSE>
        <OZ-SERVER-FRAMEWORK-SERVLET-REPORT
            ...
            USE-SERVERBIND="TRUE"
```

② spmgr.properties 설정 파일
오즈 서버의 spmgr.properties 파일을 텍스트 편집기로 열고 아래와 같이 설정합니다.
allow_exportbind_service을 true로 설정해야 서버바인딩 기능을 사용할 수 있습니다.
예) C:₩Tomcat₩webapps₩oz80₩WEB-INF₩conf₩spmgr.properties

```
allow_exportbind_service=true
exportbind_sessionkey=
```

③ 보고서 업로드

서버 바인딩 할 보고서 파일(ozr, odi)을 리파지토리 서버에 업로드합니다.

예) C:₩Tomcat₩webapps₩oz80₩WEB-INF₩repository_files₩test.ozr

3. 개발 소스

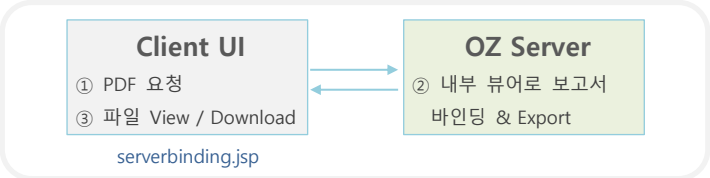
서버바인딩으로 export된 결과를 웹 브라우저에 보여주거나 다운로드하는 것은 오즈 서버 URL로 직접 요청해도 되나, 서버에 파일로 저장하려면 해당 로직이 들어간 jsp가 있어야 합니다. 아래 예제에서 export.jsp, third.jsp, oz_serverbind.jsp가 그런 기능을 수행합니다. **이러한 로직이 들어간 jsp는 반드시 오즈 서버 아래 있어야 동작됩니다.**

예제2는 서버바인딩 초기 소스입니다. 서버에 파일 생성시 예제3 또는 예제4의 소스를 적용하시기 바랍니다.

■ 패러미터 설명

구분	값	설명
ozserverexport	보고서를 서버에서 바인딩하여 파일로 저장하는 기능을 사용하도록 설정합니다.	
	true	보고서를 서버에서 바인딩하여 파일로 저장하는 기능 사용함 true 이외의 값은 에러 처리됩니다.
filename	보고서를 서버에서 바인딩하여 파일로 저장 시 파일 이름을 설정합니다. ozserverexport=true일 경우에는 반드시 설정하여야 하며, 파일 포맷을 포함한 파일 이름으로 설정합니다. 사용 가능한 파일 포맷은 pdf, xls, doc, ppt, html, csv, txt, tif, hwp 등 입니다. html로 설정 시 mht 포맷으로 저장되며, tif 파일은 무조건 하나의 파일로 생성됩니다.	
exportview	익스포트된 파일을 웹 브라우저에 바로 표시할지 여부를 설정합니다.	
	true	익스포트된 파일을 웹 브라우저에 바로 표시함 값을 true로 설정하여도 웹 브라우저에서 바로 표시할 수 없는 파일 포맷은 저장 다이얼로그가 실행됩니다.
	false	익스포트된 파일을 클라이언트에 저장할 수 있도록 저장 다이얼로그 실행함 (기본 값)
gzipcontent	서버 바인딩을 요청한 웹 브라우저가 GZip 압축 기능을 지원하는 경우 GZip 형태로 압축하여 전송할지 여부를 설정합니다.	
	true	압축함 (기본 값) 값을 true로 설정하여도 웹 브라우저가 GZip 압축 기능을 지원하지 않는 경우에는 압축되지 않습니다. 파일 포맷 중 HTML, MHT, DOC, PPT, XLS, HWP, CSV, TXT 형태만 지원하며, PDF, TIF 포맷은 파일 자체가 압축된 파일이므로 지원하지 않습니다.
	false	압축 안 함

■ 예제1 : PDF 생성 후 웹 브라우저에서 열기



다음 예제는 Search 버튼을 클릭하면 서버에서 생성된 PDF 데이터가 웹 브라우저에 표시되거나 다운로드 됩니다. 서버에 PDF 파일이 저장되진 않습니다.

- 관련 파일 : serverbinding.jsp

① 오즈 서버 아래 다음과 같은 jsp를 작성합니다. 보고서 이름과 패러미터는 환경에 맞게 수정합니다.

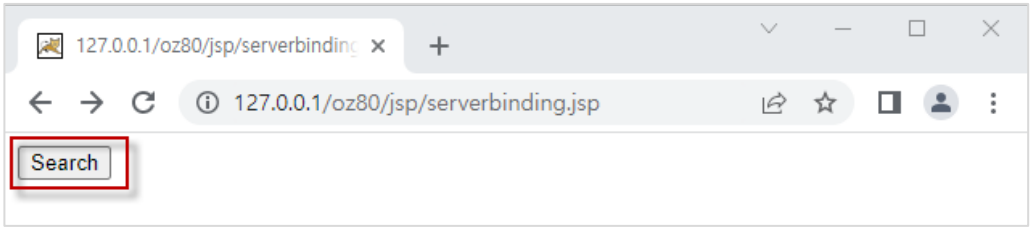
예) /oz80/jsp/serverbinding.jsp

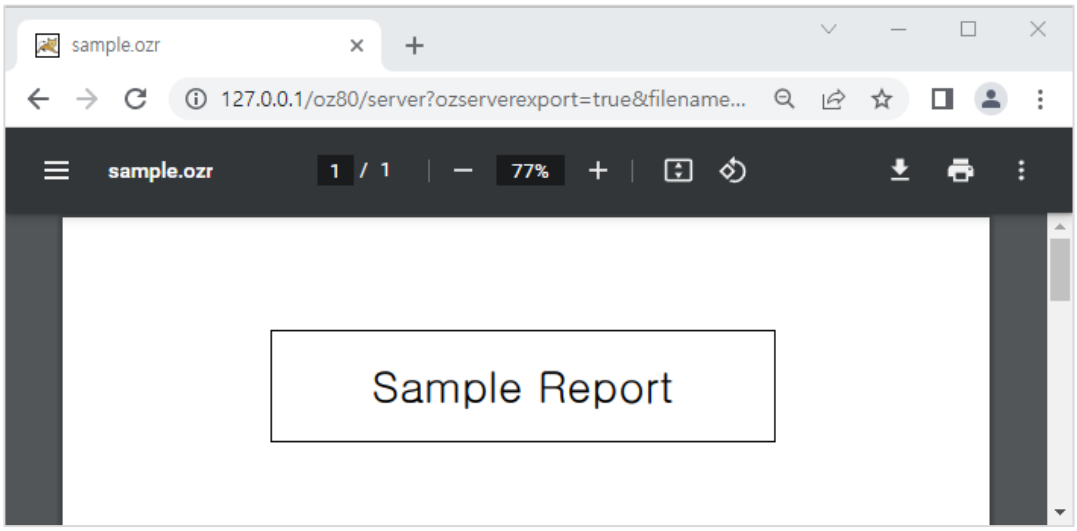
```
<html>
<body>
<form name="form1" method="get" action="/oz80/server" accept-charset="ks_c_5601-1987">
  <input type="hidden" name="ozserverexport" value="true">
  <input type="hidden" name="exportview" value="true">
  <input type="hidden" name="filename" value="sample.pdf">
  <input type="hidden" name="pdf.fontembedding" value="true">
  <input type="hidden" name="pdf.fontembedding_subset" value="true">
  <input type="hidden" name="connection.reportname" value="sample.ozr">
  <!-- 폼 패러미터 -->
  <input type="hidden" name="connection.pcount" value="1">
  <input type="hidden" name="connection.args1" value="year=2022">
  <!-- ODI 패러미터 -->
  <input type="hidden" name="odi.odinames" value="Customer">
  <input type="hidden" name="odi.Customer.pcount" value="2">
  <input type="hidden" name="odi.Customer.args1" value="Gender=F">
  <input type="hidden" name="odi.Customer.args2" value="MemberCard=Golden">
  <input type="submit" name="submit" value="Search">
</form>
</body>
</html>
```

- pdf로 export하는 경우 폰트포함 옵션(pdf.fontembedding=true)을 설정해야 다른 폰트로 보여지는 것을 방지할 수 있습니다.

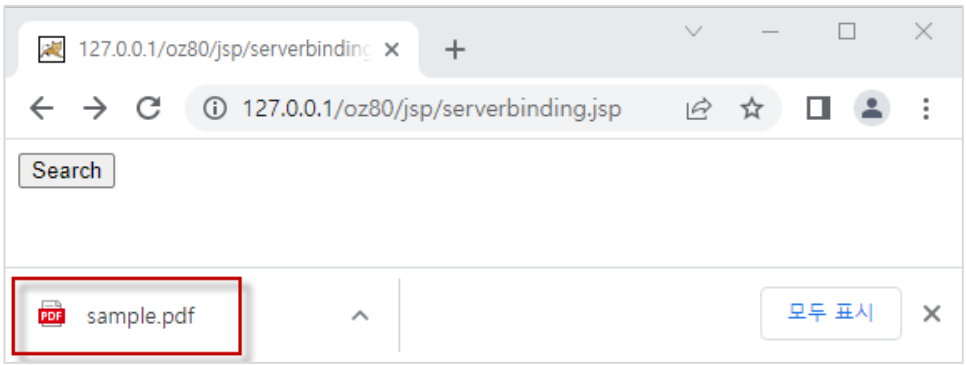
② 웹 브라우저 주소창에 위에서 작성한 jsp 경로를 입력합니다. 서버에서 받은 PDF는 웹 브라우저의 PDF 뷰어를 통해 보여집니다.

예) http://127.0.0.1/oz80/jsp/serverbinding.jsp





exportview 값을 false로 넘기면 pdf가 보여지지 않고 파일로 다운로드 됩니다.



■ 예제2 : PDF 생성 후 서버에 저장

다음 예제는 Save 버튼을 클릭하면 서버에 PDF 파일을 생성합니다.

- 관련 파일 : serverbinding_export.jsp, export.jsp
- 페이지별로 파일이 생성되는 jpg 같은 포맷의 경우 export.jsp는 첫 페이지만 파일 생성이 됩니다. 예제3의 third.jsp는 이러한 부분이 개선된 소스입니다.

① 오즈 서버 아래 serverbinding_export.jsp를 작성합니다. 보고서 이름과 패러미터는 환경에 맞게 수정합니다.

서버에 파일을 생성하려면 오즈 서블릿 (/oz80/server)로 action을 보내지 말고, 서버에 파일을 생성하는 로직을 포함한 jsp로 action을 보내야 합니다.

예제에서는 serverbinding_export.jsp 화면 소스에서 export.jsp 로직 소스를 호출합니다.

예) /oz80/jsp/serverbinding_export.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8"%>
<html>
<body>
<form name="form1" method="get" action="./export.jsp" accept-charset="ks_c_5601-1987">
  <input type="hidden" name="ozserverexport" value="true">
  <input type="hidden" name="exportview" value="true">
  <input type="hidden" name="download" value="true">
  <input type="hidden" name="filename" value="test1.pdf">
  <input type="hidden" name="connection.reportname" value="test.ozr">
  <input type="hidden" name="connection.pcount" value="1">
  <input type="hidden" name="connection.args1" value="year=2022">
  <input type="submit" name="submit" value="Save">
</form>
</body>
```

```
</html>
```

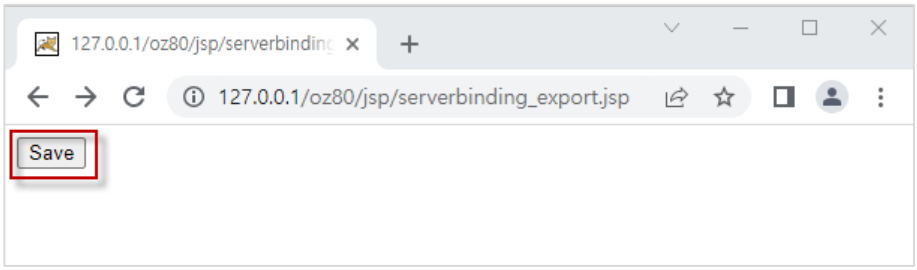
② export.jsp 파일을 열고 exportResultPath 에 파일 저장 경로를 설정합니다. 또한 필요한 뷰어 패러미터를 param.put 함수로 추가할 수 있습니다.

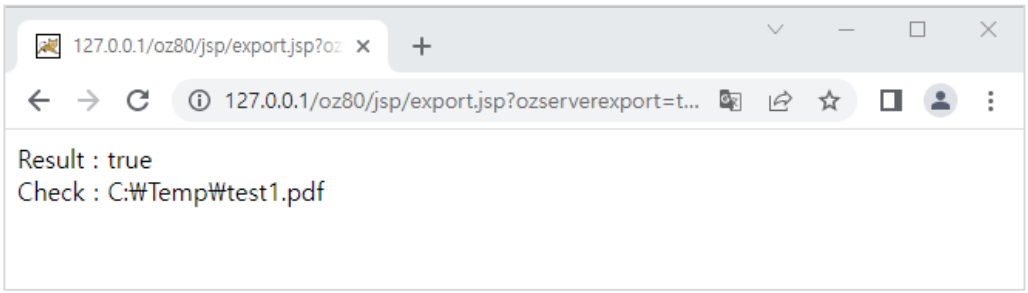
예) /oz80/jsp/export.jsp

```
String exportResultPath = "C:₩₩Temp"; // 서버에 export한 파일을 저장할 경로

// param.put 함수로 필요한 뷰어 패러미터 추가
param.put("pdf.fontembedding", "true");
param.put("pdf.fontembedding_subset", "true");
```

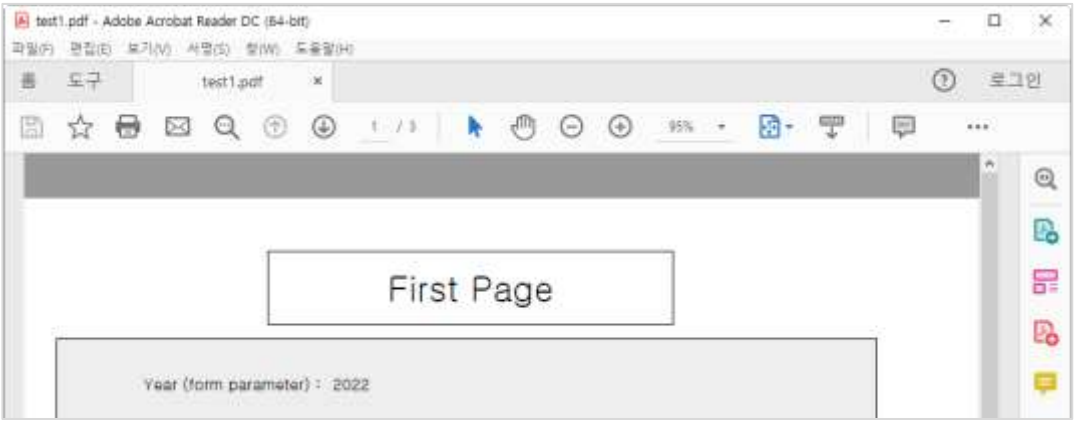
③ 웹 브라우저 주소창에 위에서 작성한 serverbinding_export.jsp 경로를 입력합니다. 'Save' 버튼을 클릭하면 서버의 지정된 경로에 파일이 생성됩니다.

예) http://127.0.0.1/oz80/jsp/serverbinding_export.jsp

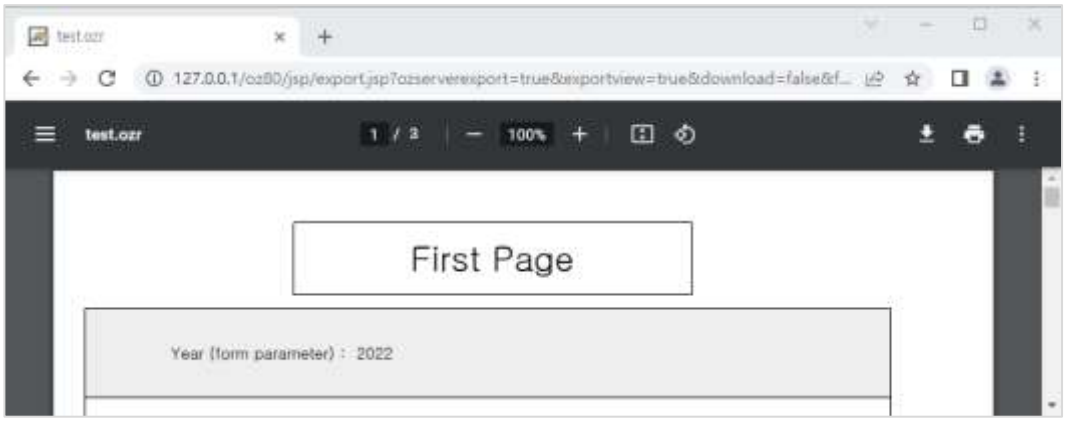




서버에 생성된 파일을 열어서 보고서 내용과 동일한지 확인합니다.

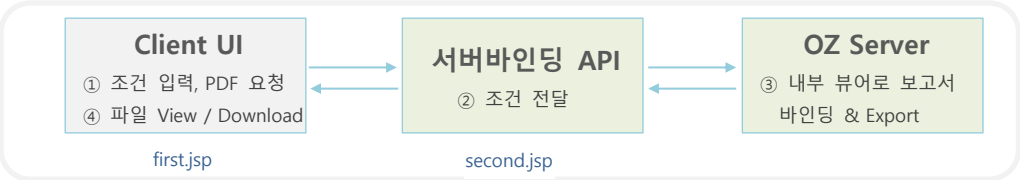


④ serverbinding_export.jsp 에서 download를 false로 설정하면 서버에 파일이 생성되지 않고 예제1 처럼 웹 브라우저 화면에 pdf가 보여집니다.



■ 예제3 : 입력한 정보로 PDF 생성

➢ 입력된 조건으로 PDF 생성 후 미리보기/다운로드



➢ 입력된 조건으로 PDF 생성 후 서버 파일 저장

다음 예제는 화면에서 입력된 조건으로 서버바인딩이 됩니다.

- 관련 파일 : first.jsp, second.jsp, third.jsp

① 오즈 서버 아래 jsp를 업로드 합니다.

예) /oz80/jsp/first.jsp, second.jsp, third.jsp

② second.jsp 파일을 열고 환경에 맞게 정보를 수정합니다.

```
service = new Service(serverUrl, "admin", "password", false, false); // 오즈 서버의 admin 계정 암호 설정

// 필요한 뷰어 패러미터 추가
ozParam += "pdf.fontembedding=true" + ozDiv;
ozParam += "pdf.fontembedding_subset=true" + ozDiv;
```

③ third.jsp 파일을 열고 환경에 맞게 정보를 수정합니다.

```
// param.put 함수로 필요한 뷰어 패러미터 추가
param.put("pdf.fontembedding", "true");
```

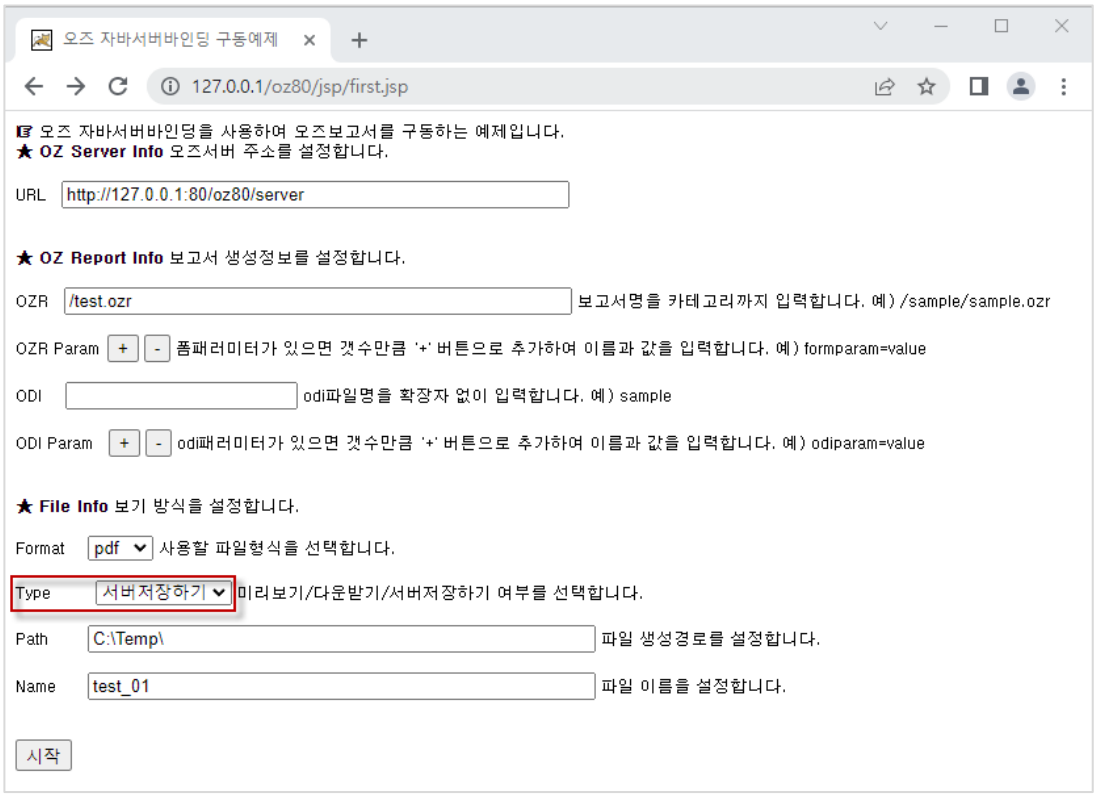
```
param.put("pdf.fontembedding_subset", "true");
```

④ 웹 브라우저에서 first.jsp 를 실행한 후 환경에 맞게 입력한 후 '시작' 버튼을 클릭합니다.

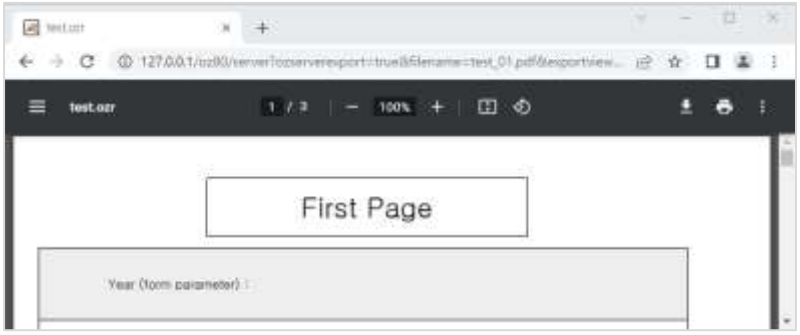
Type에 따라 다른 jsp가 수행됩니다.

- '미리보기/다운받기' 선택 : first.jsp -> second.jsp 실행
- '서버저장하기' 선택 : first.jsp -> third.jsp 실행

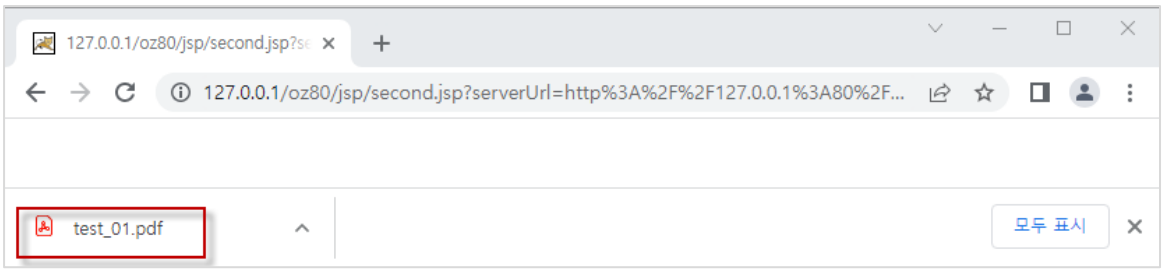
예) http://127.0.0.1/oz80/jsp/first.jsp



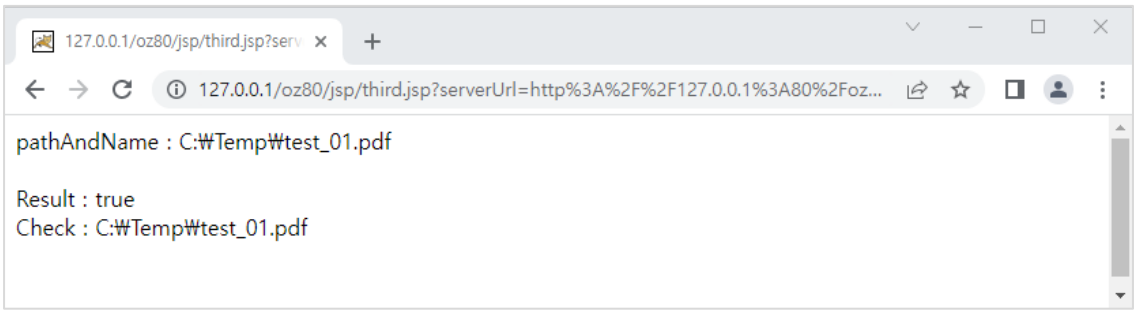
<Type이 '미리보기' 인 경우 결과 화면>

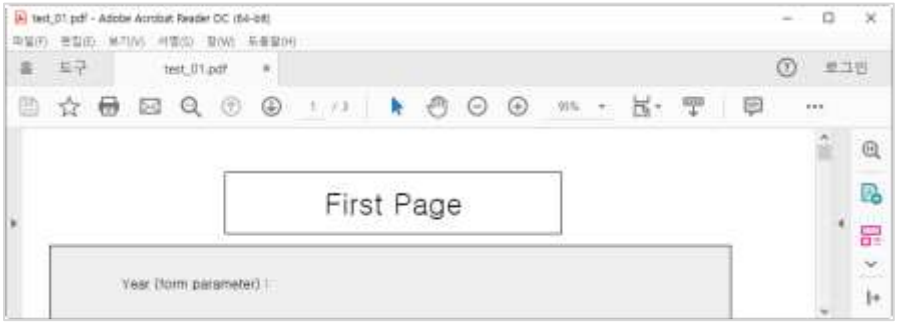


<Type이 '다운받기' 인 경우 결과 화면>



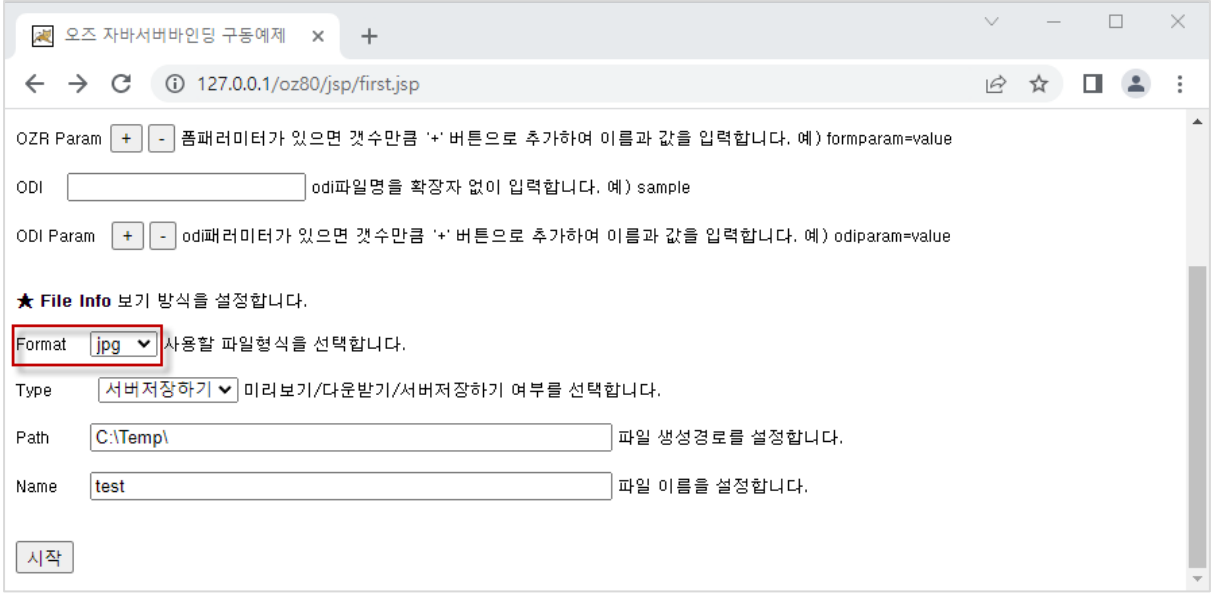
<Type이 '서버저장하기' 인 경우 결과 화면>

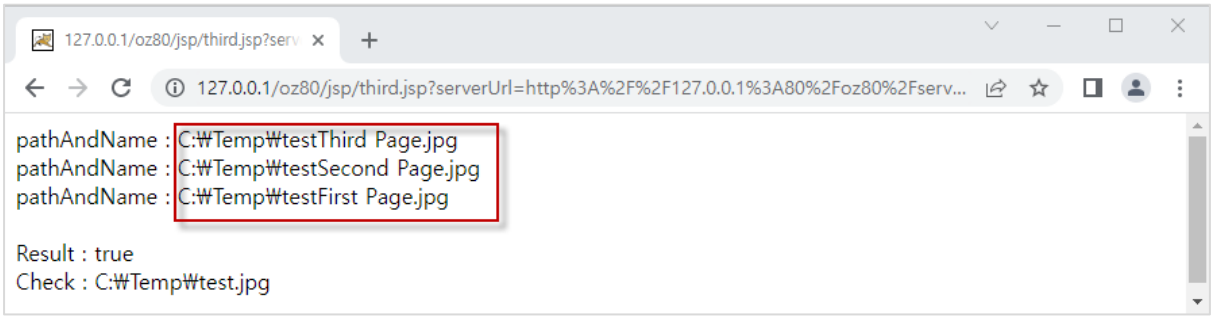




<Format이 jpg인 경우 결과 화면>

jpg의 경우 페이지 별로 파일이 생성됩니다.


오즈 자바서버바인딩 구동예제
127.0.0.1/oz80/jsp/first.jsp
OZR Param + - 폼패러미터가 있으면 갯수만큼 '+' 버튼으로 추가하여 이름과 값을 입력합니다. 예) formparam=value
ODI odi파일명을 확장자 없이 입력합니다. 예) sample
ODI Param + - odi패러미터가 있으면 갯수만큼 '+' 버튼으로 추가하여 이름과 값을 입력합니다. 예) odiparam=value
★ File Info 보기 방식을 설정합니다.
Format jpg 사용할 파일형식을 선택합니다.
Type 서버저장하기 미리보기/다운받기/서버저장하기 여부를 선택합니다.
Path C:\Temp\ 파일 생성경로를 설정합니다.
Name test 파일 이름을 설정합니다.
시작


127.0.0.1/oz80/jsp/third.jsp?serv
127.0.0.1/oz80/jsp/third.jsp?serverUrl=http%3A%2F%2F127.0.0.1%3A80%2Foz80%2Fserv...
pathAndName : C:₩Temp₩testThird Page.jpg
pathAndName : C:₩Temp₩testSecond Page.jpg
pathAndName : C:₩Temp₩testFirst Page.jpg
Result : true
Check : C:₩Temp₩test.jpg

■ 예제4 : 멀티보고서 파일 생성

다음은 멀티보고서(여러 ozr을 한번에 바인딩)를 파일로 저장하는 예제입니다.

- savePDF_multi.jsp : 서버바인딩 호출하는 페이지로 setParam() 함수에 뷰어패러미터 정의
- oz_serverbind.jsp : 서버바인딩 로직이 구현되어 있는 소스

미리보기와 동일하게 뷰어패러미터 정의에 따라 여러 파일 생성도 가능합니다.

export.saveonefile=true 를 주면 멀티보고서가 한 파일로 생성되며,

아래와 같이 주면 각각 파일로 저장됩니다.

export.savemultidoc=true

export.path=저장경로

export.filename=파일명.pdf

① 오즈 서버 아래 jsp를 업로드 합니다.

예) /oz80/jsp/savePDF_multi.jsp, oz_serverbind.jsp

② savePDF_multi.jsp 파일을 열고 setParam() 함수의 보고서 이름과 패러미터 값을 수정합니다.

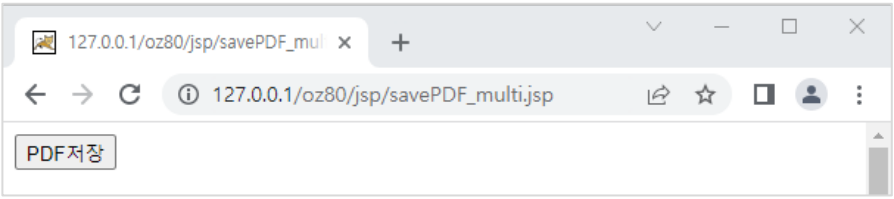
- 파일 포맷 : export.format
- 파일 경로 : export.path
- 파일 이름 : export.filename

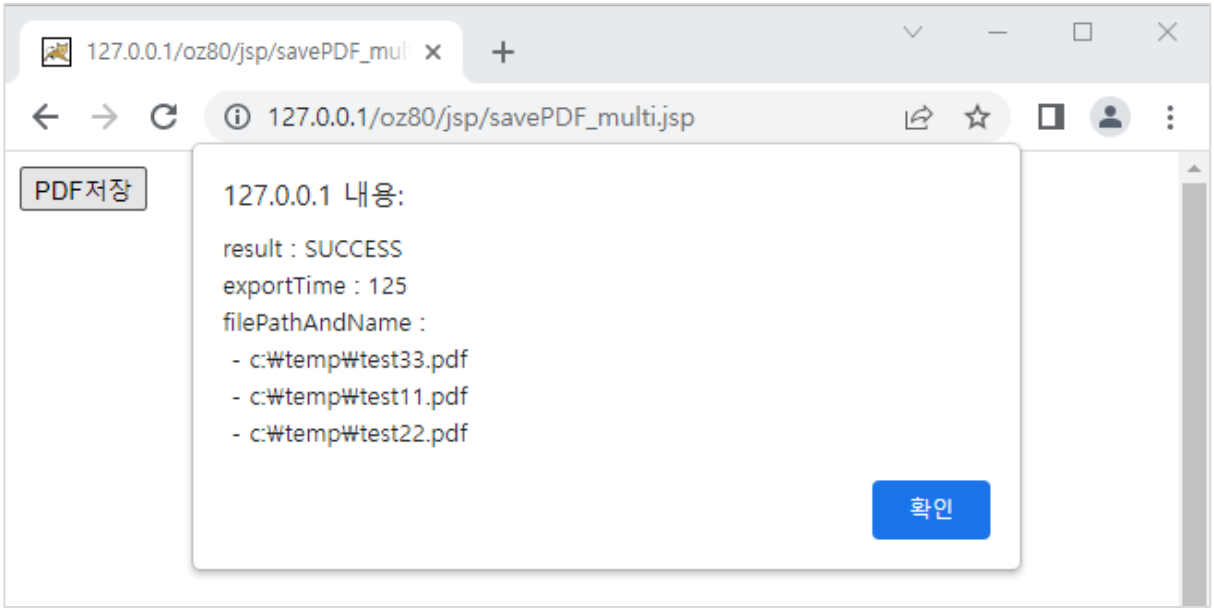
```
function setParam(){
        var ozsep = ";OZ;";
        ozParamStr = "";
        // 저장할 포맷, 저장옵션
        ozParamStr += "viewer.childcount=2"+ozsep;
        ozParamStr += "global.inheritparameter=true"+ozsep; //child에 패러미터 상속
```

```
            ozParamStr += "export.format=pdf"+ozsep;
            ozParamStr += "export.savemultidoc=true"+ozsep;
            ozParamStr += "pdf.fontembedding=true"+ozsep;
            ozParamStr += "pdf.fontembedding_subset=true"+ozsep;

            // 첫번째 보고서 이름,패러미터,저장경로
            ozParamStr += "connection.reportname=test1.ozr"+ozsep;
            ozParamStr += "connection.pcount=1"+ozsep;
            ozParamStr += "connection.args1=year=2010"+ozsep;
            ozParamStr += "export.path=c:₩₩temp₩₩"+ozsep;
            ozParamStr += "export.filename=test11.pdf"+ozsep;

            // 두번째 보고서 이름,패러미터,저장경로
            ozParamStr += "child1.connection.reportname=test2.ozr"+ozsep;
            ozParamStr += "child1.connection.pcount=2"+ozsep;
            ozParamStr += "child1.connection.args1=year=2015"+ozsep;
            ozParamStr += "child1.connection.args2=month=3"+ozsep;
            ozParamStr += "child1.export.path=c:₩₩temp₩₩"+ozsep;
            ozParamStr += "child1.export.filename=test22.pdf"+ozsep;
...
}
```

③ 웹 브라우저에서 savePDF_multi.jsp 를 실행하고 'PDF저장' 버튼을 클릭합니다.

예) http://127.0.0.1/oz80/jsp/savePDF_multi.jsp





④ 서버에 생성된 파일을 열어서 보고서 내용과 동일한지 확인합니다.

■ 예제5 : 서버바인딩 서블릿

위의 예제2의 jsp를 서블릿으로 만든 예제입니다.

- ServerBindTest.jsp : 서버바인딩 호출하는 페이지
- ServerBindServlet.java : 서버바인딩 로직이 구현되어 있는 소스

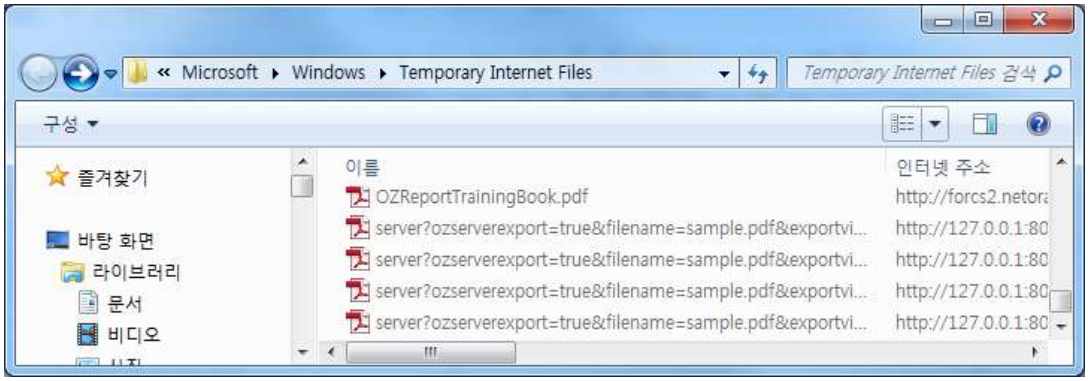
4. 디버깅

■ 참고 사항

- 대용량 보고서는 서버 PC의 메모리 등 리소스를 많이 사용하게 되므로, 데이터 양에 따라 에러가 발생할 수 있습니다.
- 에러 발생 시 웹 브라우저에서는 500 에러가 발생합니다.
- 본 기능을 사용 중일 경우 오즈 서버 로그에 'OZServerExport'라고 표시됩니다.

```
2022-04-21 14:27:06,587 [http-nio-80-exec-9] DEBUG OZServerExport - OZ Server Export Start.
```

- 파일 이름을 html 포맷으로 설정 시 exportview=true인 경우 웹 브라우저에 이미지가 표시되지 않을 수도 있습니다.
- filename으로 지정한 파일은 서버에 생성되지 않습니다. 실행한 사용자 PC의 인터넷 임시 파일 경로에 임의의 이름으로 생성됩니다.



■ 예제1 서버 로그

위의 예제1이 정상적으로 수행될 때 오즈 서버 로그 입니다. export가 안되거나 export 결과가 이상이 있는 경우 로그 정보를 보고 디버깅 합니다. 패러미터 값이 넘어 갔는지 로그를 보고 확인할 수 있습니다.

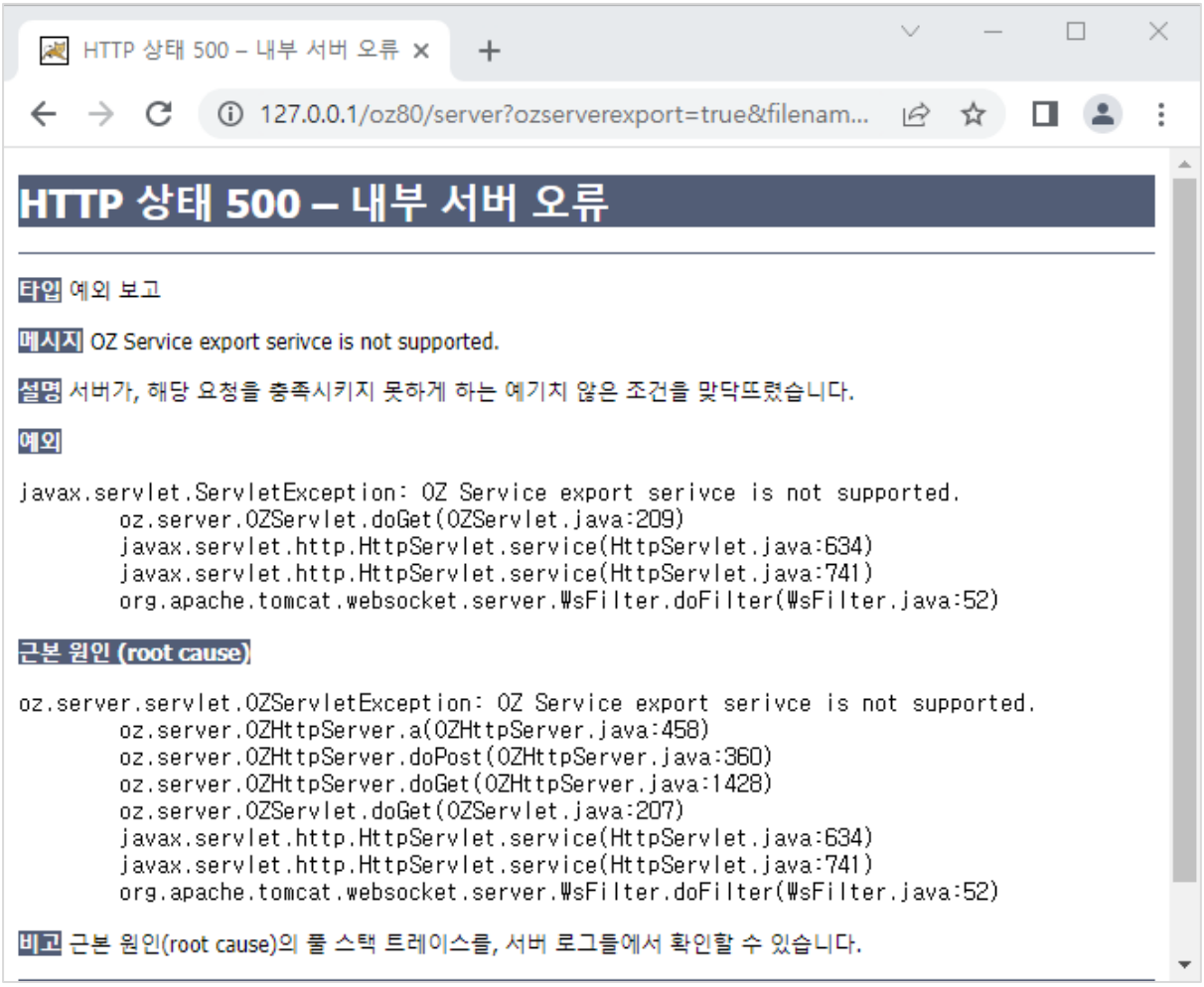
예) /oz80/WEB-INF/logs/server.log

```
2022-04-21 14:27:06,585 [http-nio-80-exec-9] INFO   server - client IP:[ 127.0.0.1] A handler is started. Active handler
count is 1
2022-04-21 14:27:06,587 [http-nio-80-exec-9] INFO   OZServerExport - REQUEST-URI : http://127.0.0.1/oz80/server
2022-04-21 14:27:06,587 [http-nio-80-exec-9] DEBUG OZServerExport - OZ Server Export Start.
2022-04-21 14:27:06,589 [http-nio-80-exec-9] DEBUG OZServerExport - Session does not check.
2022-04-21 14:27:06,589 [http-nio-80-exec-9] DEBUG OZServerExport - pdf.fontembedding=true
2022-04-21 14:27:06,590 [http-nio-80-exec-9] DEBUG OZServerExport - pdf.fontembedding_subset=true
2022-04-21 14:27:06,592 [http-nio-80-exec-9] DEBUG OZServerExport - connection.reportname=sample.ozr
2022-04-21 14:27:06,593 [http-nio-80-exec-9] DEBUG OZServerExport - submit=Search
2022-04-21 14:27:06,596 [http-nio-80-exec-9] DEBUG OZServerExport - Export start.
2022-04-21 14:27:06,598 [http-nio-80-exec-9] INFO   OZServerExport - ****************************
2022-04-21 14:27:06,600 [http-nio-80-exec-9] INFO   OZServerExport - Forcs Co.,Ltd. (OZ)
2022-04-21 14:27:06,600 [http-nio-80-exec-9] INFO   OZServerExport - Version - 80,2022,0421,100
2022-04-21 14:27:06,601 [http-nio-80-exec-9] INFO   OZServerExport - Name Space - OZ 80 001
2022-04-21 14:27:06,601 [http-nio-80-exec-9] INFO   OZServerExport - Compatible common protocol : 2022
2022-04-21 14:27:06,602 [http-nio-80-exec-9] INFO   OZServerExport - Release mode - e
2022-04-21 14:27:06,603 [http-nio-80-exec-9] INFO   OZServerExport - Java Version - 16.0.2, Windows 10, ko
2022-04-21 14:27:06,603 [http-nio-80-exec-9] INFO   OZServerExport - ****************************
2022-04-21 14:27:06,605 [http-nio-80-exec-9] DEBUG OZServerExport - ------------------------------------------------------
-------------
2022-04-21 14:27:06,607 [http-nio-80-exec-9] DEBUG OZServerExport - User Defined Parameters
2022-04-21 14:27:06,607 [http-nio-80-exec-9] DEBUG OZServerExport -
2022-04-21 14:27:06,608 [http-nio-80-exec-9] DEBUG OZServerExport -
repository_agent.ozserver.servlet=ozserver
connection.reportname=sample.ozr
repository_agent.type=FROM_OZSERVER
print.useprogressbar=false
export.format=pdf
export.path=C:/
export.filename=sample.pdf
export.mode=silent
export.confirmsave=false
export.saveonefile=true
export.useprogressbar=false
information.debug=debug
viewer.isframe=false
viewer.mode=export
viewer.useprogressbar=false
viewer.allowmultiframe=false
viewer.showerrormessage=false
pdf.fontembedding=true
pdf.fontembedding_subset=true
tiff.savemultipage=true
svg.savemultipage=true
mail.showbodyeditor=false
```

```
2022-04-21 14:27:06,610 [http-nio-80-exec-9] DEBUG OZServerExport - ----------------------------------------------------
-------------
2022-04-21 14:27:06,611 [http-nio-80-exec-9] INFO  OZServerExport - 보고서 탬플릿 생성 정보 : Servlet
2022-04-21 14:27:06,612 [http-nio-80-exec-9] INFO  OZServerExport - 서버로의 연결을 준비하고 있습니다.
2022-04-21 14:27:06,613 [http-nio-80-exec-9] INFO  OZServerExport - 서버와 성공적으로 연결되었습니다.
2022-04-21 14:27:06,614 [http-nio-80-exec-9] INFO  OZServerExport - 서버로부터 보고서 폼을 읽고 있습니다.
2022-04-21 14:27:06,615 [http-nio-80-exec-9] INFO  MessageHandler - Start Message Handing
2022-04-21 14:27:06,615 [http-nio-80-exec-9] DEBUG MessageHandler -
[version:10001][className:oz.framework.cp.message.repository.OZRepositoryRequestUserLogin]
2022-04-21 14:27:06,616 [http-nio-80-exec-9] DEBUG MessageHandler - Request Message starts fetching for Response
2022-04-21 14:27:06,616 [http-nio-80-exec-9] DEBUG OZRepositoryRequestUserLogin - Reading Message is successfully
completed
2022-04-21 14:27:06,617 [http-nio-80-exec-9] DEBUG OZRepositoryRequestUserLogin - Function Type [176 :
LOGIN_TO_SERVER]
2022-04-21 14:27:06,618 [http-nio-80-exec-9] INFO  MessageHandler - End Message Handing
2022-04-21 14:27:06,619 [http-nio-80-exec-9] INFO  MessageHandler - Start Message Handing
2022-04-21 14:27:06,622 [http-nio-80-exec-9] DEBUG MessageHandler -
[version:10001][className:oz.framework.cp.message.repositoryex.OZRepositoryRequestItem]
2022-04-21 14:27:06,624 [http-nio-80-exec-9] DEBUG MessageHandler - Request Message starts fetching for Response
2022-04-21 14:27:06,625 [http-nio-80-exec-9] DEBUG OZRepositoryRequestItem - Reading Message is successfully
completed
2022-04-21 14:27:06,626 [http-nio-80-exec-9] DEBUG OZRepositoryRequestItem - Function Type [256 :
ACCESS_GET_ITEMS]
2022-04-21 14:27:06,628 [http-nio-80-exec-9] DEBUG OZStream - write Item zip size = 1107
2022-04-21 14:27:06,630 [http-nio-80-exec-9] INFO  MessageHandler - End Message Handing
2022-04-21 14:27:06,631 [http-nio-80-exec-9] INFO  OZServerExport - 보고서 템플릿을 생성하고 있습니다.
2022-04-21 14:27:06,634 [http-nio-80-exec-9] INFO  OZServerExport - 서버로부터 데이터를 읽고 있습니다.
2022-04-21 14:27:06,641 [http-nio-80-exec-9] DEBUG OZServerExport - Export finish.
2022-04-21 14:27:06,642 [http-nio-80-exec-9] DEBUG OZServerExport - OZ Server Export finish.
```

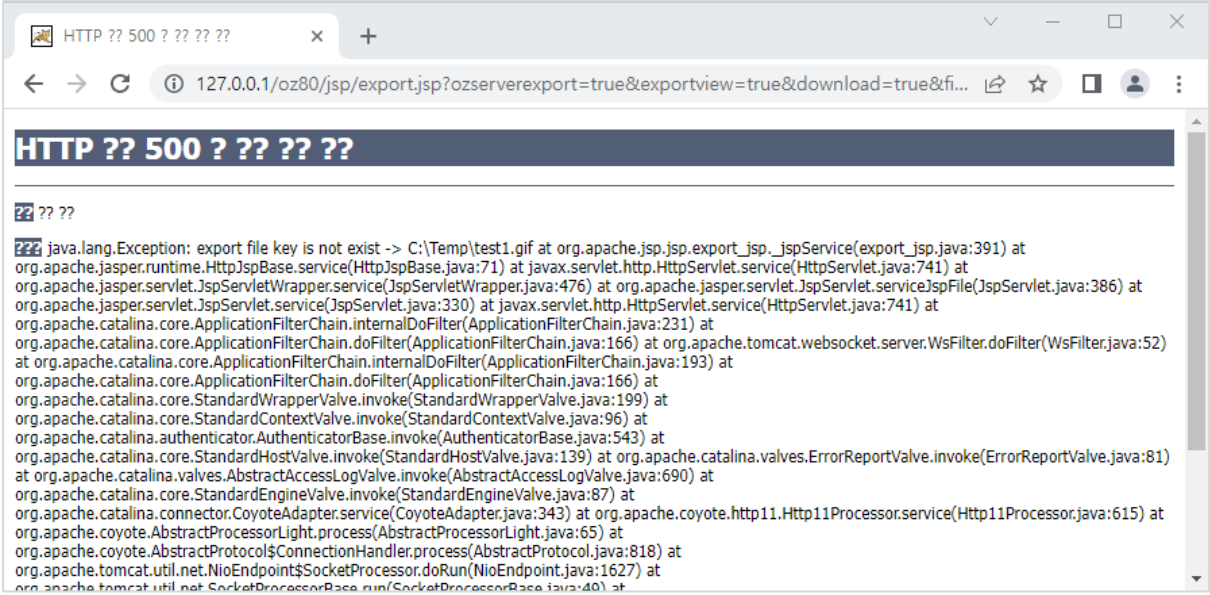
■ OZ Service export serivce is not supported 에러

다음과 같이 "OZ Service export serivce is not supported." 에러가 발생하는 경우 spmgr.properties 에 환경 설정을 했는지 확인해야 합니다.



■ export file key is not exist 에러

export.jsp에 정의되지 않은 확장자로 요청한 경우 다음과 같이 "export file key is not." 에러가 발생할 수 있습니다.


HTTP ?? 500 ? ?? ?? ??
127.0.0.1/oz80/jsp/export.jsp?ozserverexport=true&exportview=true&download=true&fi...
HTTP ?? 500 ? ?? ?? ??
?? ?? ??
??? java.lang.Exception: export file key is not exist -> C:\Temp\test1.gif at org.apache.jsp.jsp.export_jsp._jspService(export_jsp.java:391) at
org.apache.jasper.runtime.HttpJspBase.service(HttpJspBase.java:71) at javax.servlet.http.HttpServlet.service(HttpServlet.java:741) at
org.apache.jasper.servlet.JspServletWrapper.service(JspServletWrapper.java:476) at org.apache.jasper.servlet.JspServlet.serviceJspFile(JspServlet.java:386) at
org.apache.jasper.servlet.JspServlet.service(JspServlet.java:330) at javax.servlet.http.HttpServlet.service(HttpServlet.java:741) at
org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:231) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:166) at org.apache.tomcat.websocket.server.WsFilter.doFilter(WsFilter.java:52)
at org.apache.catalina.core.ApplicationFilterChain.internalDoFilter(ApplicationFilterChain.java:193) at
org.apache.catalina.core.ApplicationFilterChain.doFilter(ApplicationFilterChain.java:166) at
org.apache.catalina.core.StandardWrapperValve.invoke(StandardWrapperValve.java:199) at
org.apache.catalina.core.StandardContextValve.invoke(StandardContextValve.java:96) at
org.apache.catalina.authenticator.AuthenticatorBase.invoke(AuthenticatorBase.java:543) at
org.apache.catalina.core.StandardHostValve.invoke(StandardHostValve.java:139) at org.apache.catalina.valves.ErrorReportValve.invoke(ErrorReportValve.java:81)
at org.apache.catalina.valves.AbstractAccessLogValve.invoke(AbstractAccessLogValve.java:690) at
org.apache.catalina.core.StandardEngineValve.invoke(StandardEngineValve.java:87) at
org.apache.catalina.connector.CoyoteAdapter.service(CoyoteAdapter.java:343) at org.apache.coyote.http11.Http11Processor.service(Http11Processor.java:615) at
org.apache.coyote.AbstractProcessorLight.process(AbstractProcessorLight.java:65) at
org.apache.coyote.AbstractProtocol$ConnectionHandler.process(AbstractProtocol.java:818) at
org.apache.tomcat.util.net.NioEndpoint$SocketProcessor.doRun(NioEndpoint.java:1627) at